

VOLUME 1 OF 6

Foundations & Reference Architecture

Security Frameworks, Agent Lifecycle Security, Runtime Isolation, and the AI Security Mesh

Document Code: EASA-01

Version: 1.0

Date: June 2026

Scope: Domains 1, 4, 7 & 20

Enterprise Agentic AI Security Architect (2026–2030) Master Research Program

Table of Contents

Executive Summary

This volume establishes the architectural foundation for everything that follows in the program. Its purpose is to answer a question that most enterprises have not yet asked formally: what does it mean to apply security architecture discipline — the kind historically reserved for networks, applications, and identity systems — to a population of autonomous software agents that plan, reason, call tools, remember, and act with delegated authority?

The honest starting position is that the agentic AI ecosystem in 2026 looks like distributed computing did in roughly 2010 and like API security did in roughly 2015: powerful, proliferating faster than governance can track it, built on protocols still hardening in production, and increasingly the subject of real, disclosed incidents rather than theoretical risk. The OWASP Top 10 for Agentic Applications, published December 2025 and already in its first major revision cycle, documents real-world incidents — prompt-injection-driven data exfiltration in enterprise copilots, tool misuse in cloud coding assistants, remote code execution in autonomous frameworks, and memory poisoning in production assistants — that mirror, almost exactly, the early years of web application security.

Architectural premise of this program

An AI agent is not a feature of an application. It is a new class of execution principal — with its own identity, its own blast radius, its own lifecycle, and its own failure modes — that must be designed, governed, and operated with the same rigor as a privileged service account, and arguably more, because it can reason about how to circumvent the controls placed around it.

This volume covers four domains that together form the architectural spine of the program: established enterprise security architecture frameworks adapted for agents (Domain 1), the security model for the agent lifecycle itself (Domain 4), the runtime and isolation layer agents execute within (Domain 7), and the security mesh pattern that ties identity, policy, memory, and observability together at scale (Domain 20). Later volumes go deep on identity, MCP, A2A, governance, and operations; this volume gives the reader the scaffolding those deep dives hang on.

Domain 1 — Security Architecture Frameworks for Agentic AI

Enterprise security architecture did not get rebuilt from scratch for cloud, and it should not be rebuilt from scratch for agents. The discipline's existing frameworks — SABSA, TOGAF, and Zachman — remain the right starting vocabulary. What changes is the population of things being modeled: a SABSA business attribute now has to account for autonomous decision-making; a TOGAF ADM cycle now has to govern a fleet of agents that can be spun up by developers outside formal change control; a Zachman cell for "Who" now has to represent non-human principals that vastly outnumber human ones.

1.1 SABSA for Agentic Systems

SABSA's core discipline — deriving security architecture from business attributes rather than from technology controls — translates directly to agents, but the attribute set needs extension. Traditional SABSA attributes (confidentiality, integrity, availability, accountability) remain necessary but not sufficient. Agentic systems require additional first-class attributes:

- **Agency boundedness** — the degree to which an agent's autonomy is constrained to a defined, auditable action space, sometimes called "Least Agency" in current OWASP guidance: autonomy should be a feature an agent earns through demonstrated reliability, not a default it starts with.
- **Reversibility** — whether an agent's actions can be undone, compensated, or contained before they propagate downstream (a database write versus a wire transfer versus a public API call each carry very different reversibility profiles).
- **Explainability under audit** — whether the chain of reasoning, tool calls, and memory reads that produced an action can be reconstructed after the fact for a regulator, auditor, or incident responder.
- **Trust derivability** — whether the agent's current trust level can be computed from verifiable signals (identity attestation, behavioral history, content provenance) rather than asserted once at deployment and never revisited.

The practical deliverable is an Agent Trust Level Derivation model: a scoring function that combines workload identity assurance, the sensitivity of the data and tools the agent can reach, its autonomy level (see Domain 14's autonomy-level taxonomy), and its historical behavioral conformance, and maps the result onto the organization's existing SABSA risk-attribute taxonomy so that agent risk can be reported in the same business language as every other risk in the enterprise risk register.

1.2 TOGAF Security Architecture and the Agent Fabric

TOGAF's Architecture Development Method (ADM) needs two structural adaptations for agentic platforms. First, Phase B (Business Architecture) and Phase D (Technology Architecture) need an explicit "Agent Fabric" viewpoint — a horizontal layer that cuts across business capabilities the way a service mesh or an integration layer does today, because agents are rarely scoped to a single business capability; they are increasingly shared infrastructure that many business processes draw on.

Second, and more consequentially, Phase G (Implementation Governance) needs a fast-path review track for agents. The standard TOGAF architecture review cadence — periodic, document-heavy, aligned to project milestones — cannot keep pace with an organization where a developer can stand up a new agent, register a new MCP tool, or grant a new scope in minutes. Most enterprises that have gotten this right have built a lightweight, automatable "Agent Architecture Compliance" gate: a machine-checkable policy (expressed in

something like Cedar or OPA — see Domain 5) that runs at registration time and blocks non-compliant agents from being provisioned, backed by a periodic human-led architecture review for anything above a defined risk threshold.

1.3 Zachman Framework Mapping

The Zachman Framework's six interrogatives map cleanly onto the agentic stack and are useful precisely because they force completeness — it is easy to design an agent platform that answers "How" (the orchestration logic) in exhaustive detail while leaving "Why" (the business justification for the agent's existence and scope) almost entirely implicit.

Interrogative	Traditional Enterprise Mapping	Agentic AI Mapping
What	Data entities, data models	Agent memory (episodic, semantic, procedural), context windows, embeddings, RAG corpora
How	Business processes, application logic	Agent reasoning loops, planning, tool invocation chains, orchestration graphs
Where	Network nodes, locations, data centers	Runtime sandboxes, MCP servers, A2A endpoints, cloud regions, edge inference
Who	Organizational units, roles, human identity	Human identities, non-human/workload identities, compound user+agent identity, agent-to-agent trust relationships
When	Business events, processing cycles	Agent lifecycle states: provisioned, active, suspended, retired; session and task lifecycles
Why	Business goals, strategy, motivation	Agent purpose statement, authorized objective, governance approval, autonomy-level justification

The deliverable from this exercise is an Agent Security Architecture Repository — a single structured inventory, ideally machine-readable, where every registered agent has a populated cell for each of the six interrogatives. In practice this becomes the backbone of the Agent Registry described in Domain 14: an agent with an undefined "Why" or an unbounded "Where" is, definitionally, ungoverned.

Comparative note: how the framework vendors are positioning this

Microsoft's Azure AI Foundry and Entra Agent ID materials describe agent governance largely through an identity-first lens (extending Entra ID's existing RBAC/PIM model to agents). Google's Agentspace and Vertex AI Agent Builder lean on Google Cloud's existing IAM and BeyondCorp zero-trust patterns. AWS's Bedrock Agents and the emerging AgentCore runtime lean on IAM roles and Verified Permissions (Cedar-based). None of the three hyperscalers has published a SABSA- or TOGAF-native reference architecture; each is extending its own cloud-native identity and policy stack outward to cover agents. This is a meaningful gap an enterprise architecture function can fill: translating each hyperscaler's agent-security primitives into the enterprise's existing SABSA/TOGAF vocabulary, rather than adopting three incompatible mental models for a multi-cloud agent estate.

1.4 Security Engineering Principles Applied to Agents

The classical security engineering principles do not need replacing; they need a precise mapping onto agent-specific mechanisms, because "defense in depth for agents" is meaningless until it specifies which layers.

Principle	Application to Agents / MCP / A2A / Memory / Runtime
Secure by Design	Agent capability manifests, tool scopes, and autonomy levels are defined and reviewed before an agent is built, not retrofitted after a pilot succeeds and gets promoted to production.
Security by Default	New agents are provisioned with zero standing tool access; capabilities are explicitly granted, never inherited from a broad template or a permissive default service account.
Defense in Depth	Independent controls at the identity layer (workload identity, OAuth scopes), the MCP/A2A protocol layer (schema validation, signed agent cards), the runtime layer (sandboxing, egress control), and the memory layer (provenance tagging, poisoning detection) — no single control is load-bearing.
Zero Trust	Every tool call, every inter-agent message, and every memory read is authenticated and authorized at the point of use; an agent's network location or prior successful actions confer no implicit trust for the next action.
Assume Breach	Architecture assumes at least one agent, one MCP server, or one A2A peer is compromised or behaving maliciously at any given time, and is designed to contain blast radius (capability scoping, egress allowlisting, circuit breakers) rather than to prevent compromise outright.
Resilience Engineering	Agent Reliability Engineering practices (Domain 19) — SLOs, error budgets, graceful degradation, circuit breakers on tool chains — treat agent misbehavior as an expected operating condition to be engineered around, not an exception.
Safety by Design	Irreversible or high-consequence actions (financial transactions, infrastructure changes, external communications) require human-in-the-loop approval gates regardless of the agent's measured trust score.
Privacy by Design	Memory and context layers classify and minimize personal data at write time; retention and right-to-erasure are lifecycle properties of agent memory, not an afterthought bolted onto a data warehouse.

1.5 Threat Modeling for Agentic Systems

Threat modeling agentic systems requires layering frameworks rather than picking one. STRIDE and PASTA remain useful for the conventional application and infrastructure surface an agent sits on top of, but neither was designed to reason about a system that forms its own intent at runtime. Three newer frameworks fill that gap, and a mature program uses all three together rather than treating them as competitors.

STRIDE and PASTA — what still applies, and where they stop

STRIDE's six categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) map cleanly onto agent identity spoofing, memory tampering, insufficient audit logging, context leakage, resource-exhaustion attacks on agent budgets, and privilege escalation through delegation chains. It is a sound checklist for the infrastructure an agent runs on. Its blind spot is cognitive: STRIDE has no native category for an agent being talked into doing something harmful through legitimate-looking input, because that is not a spoofing or tampering attack against the system — it is a successful, well-formed instruction that happens to be malicious. PASTA's seven-stage, risk-centric process (defining business objectives, technical scope, decomposing the application, analyzing threats, identifying vulnerabilities, modeling attacks, and quantifying risk) is valuable for tying agent risk back to business impact, but suffers the same blind spot at the decomposition stage unless explicitly extended.

MITRE ATT&CK and MITRE ATLAS

ATT&CK remains the right taxonomy for everything that happens once an agent (or the infrastructure underneath it) is compromised and an adversary is operating within the environment — lateral movement, credential access, persistence. ATLAS extends this specifically to the ML/AI attack surface: adversarial inputs, model extraction, training-data poisoning, and (in its more recent additions) agent-specific tactics. A mature threat model cross-references ATLAS techniques against the agent's specific architecture: which ATLAS techniques are even reachable given this agent's tool access, memory configuration, and autonomy level.

CSA MAESTRO — the agent-native framework

MAESTRO (Multi-Agent Environment, Security, Threat, Risk, and Outcome), published by the Cloud Security Alliance, is purpose-built for agentic systems and has become the closest thing to a standard agent threat-modeling methodology in 2026. It decomposes an agentic system into seven layers and requires the threat modeler to work through both traditional and agentic threats at each layer, then explicitly analyze cross-layer threats — because the most damaging agent failures rarely stay contained to one layer.

Layer	What It Covers	Representative Threats
L1 — Foundation Models	The base/fine-tuned LLMs providing reasoning and tool-use capability	Adversarial inputs, model extraction, training-data and fine-tuning backdoors
L2 — Data Operations	Data pipelines, labeling, embeddings, RAG ingestion	Data poisoning, embedding manipulation, pipeline tampering
L3 — Agent Frameworks	Orchestration logic, planning, decision-making (LangGraph, AutoGen, etc.)	Goal hijacking, insecure tool integration, prompt injection propagation
L4 — Deployment	Containers, cloud hosting, network	Misconfiguration, container escape, insecure

Layer	What It Covers	Representative Threats
& Infrastructure	configuration	secrets handling
L5 — Evaluation & Observability	Testing, monitoring, logging, audit trails	Blind spots in monitoring, evaluation gaming, log tampering
L6 — Security & Compliance (vertical)	Cross-cutting controls applied at every layer	Inconsistent policy enforcement across layers, compliance drift
L7 — Agent Ecosystem	Multi-agent interaction, marketplaces, A2A/MCP discovery	Agent impersonation, marketplace manipulation, collusion, supply-chain compromise via third-party agents

MAESTRO's real value is forcing analysis of cross-layer cascades: a poisoned embedding at L2 reaches an agent's planning logic at L3, which selects a tool at L4 that has broader permissions than the original task required, and the resulting action is never flagged because the observability layer (L5) was only watching for anomalies in single-agent behavior, not for an L7-level pattern of one compromised agent influencing a peer through A2A messaging. No single-layer review catches this; only an explicit cross-layer pass does.

Producing the four required threat models

This program calls for fully worked threat models, using MAESTRO as primary structure cross-referenced against ATLAS and STRIDE, for four representative system types. Each should be built as a standalone artifact (data flow diagram, trust boundaries, layer-by-layer threat enumeration, and a prioritized control list) and reviewed at least annually or on material architecture change:

1. Single Agent — a bounded, single-purpose agent (e.g., a customer support agent with read access to a CRM and a knowledge base). Establishes the baseline pattern and the minimum control set every agent should carry regardless of complexity.
2. Multi-Agent System — a planner/orchestrator coordinating two or more specialist sub-agents (e.g., a research agent delegating to a web-search agent and a code-execution agent). Surfaces delegation-abuse, state-poisoning, and goal-hijack-propagation risks absent in the single-agent case.
3. MCP Ecosystem — an enterprise with a central MCP gateway fronting dozens of internal and third-party MCP servers. Surfaces tool poisoning, schema manipulation, and tenant-isolation risks (full detail in Volume 2).
4. Banking AI Platform — a regulated, high-consequence composite: agents with access to payment rails, customer PII, and external A2A counterparties (correspondent-bank agents, payment-network agents). This is the reference model used throughout the program's banking-grade reference architecture (Volume 6) and should incorporate every control surfaced by the prior three models plus regulatory-specific constraints (DORA, PCI DSS, SOC 2 — see Volume 3).

Domain 4 — Agentic AI Architecture Security

Domain 4 takes the threat-modeling output from Domain 1 and turns it into a concrete control architecture, organized around the canonical agent lifecycle and the realities of multi-agent orchestration frameworks now in production use.

4.1 Agent Lifecycle Security

Every agentic interaction, regardless of framework, decomposes into the same five-stage lifecycle: a user or trigger initiates a request, a planner interprets intent and forms a plan, an agent (or agents) executes steps of that plan, tools are invoked to take action or retrieve information, memory is read and written throughout, and actions land in the world. Each stage has a distinct threat profile, a distinct control set, and a distinct audit requirement.

Stage	Primary Threats	Primary Controls	Audit Requirement
User → Planner	Prompt injection (direct and indirect via RAG/email/documents), intent manipulation	Input sanitization, content provenance tagging, instruction-vs-data separation	Full input capture with provenance metadata
Planner → Agent	Goal hijacking, plan tampering, excessive-scope plan generation	Plan validation against policy before execution, bounded planning depth, human approval for high-risk plans	Plan diff logging (intended vs. executed)
Agent → Tool	Tool misuse, parameter injection, excessive permissions, toxic tool combinations	Capability-scoped tokens, schema validation at invocation, tool allowlisting, approval workflows for sensitive calls	Every tool call logged with parameters, caller identity, and policy decision
Tool → Memory	Memory/context poisoning, cross-tenant memory leakage, unauthorized persistence	Write-time classification, provenance tagging, tenant isolation, retention policy enforcement	Memory mutation log with source attribution
Memory/Tool → Action	Irreversible or high-consequence actions taken without sufficient review	Reversibility classification, human-in-the-loop gates for irreversible actions, rate limiting, circuit breakers	Immutable action log, tamper-evident, tied to originating session

4.2 Multi-Agent Orchestration Frameworks and Their Threat Surface

The production orchestration landscape has consolidated around a handful of frameworks, each with a distinct trust model that materially changes the threat surface:

- **LangGraph** — graph-based orchestration with explicit state; the principal risk is state poisoning, since shared graph state is often trusted implicitly by downstream nodes.

- **CrewAI** — role-based crews with delegation between agents; delegation abuse is the dominant risk, where a low-privilege agent convinces a higher-privilege peer to act on its behalf.
- **Microsoft Agent Framework (the successor combining Semantic Kernel and AutoGen patterns)** — enterprise-oriented orchestration with native Entra Agent ID integration; the main architectural advantage is identity-aware delegation, but it also means identity misconfiguration has an outsized blast radius.
- **AutoGen** — conversational multi-agent patterns; infinite-loop and goal-drift risk is highest here because agent-to-agent conversations can continue without a hard termination condition unless explicitly bounded.
- **Semantic Kernel Agents** — plugin-centric orchestration; tool/plugin supply-chain risk dominates, since the framework's extensibility model is built around third-party plugin ecosystems.

The five threats common across all five frameworks — agent collusion, goal hijacking, delegation abuse, state poisoning, and infinite loops — are precisely the threats ASI07 (Insecure Inter-Agent Communication), ASI08 (Cascading Failures), and ASI10 (Rogue Agents) in the OWASP Top 10 for Agentic Applications are designed to catch. Volume 4 covers detection and response for these in depth; the architectural control here is structural: bound every multi-agent conversation with a maximum turn count, a maximum cost budget, and a maximum wall-clock duration, enforced outside the orchestration framework itself (so a compromised or malfunctioning orchestrator cannot disable its own circuit breaker).

Domain 7 — Agent Runtime Security

If identity answers "who is acting" and policy answers "what are they allowed to do," the runtime layer answers the question that actually contains the blast radius when the first two fail: what can this process physically reach? Runtime isolation is the last line of defense, and for agents that execute generated code or shell commands, it is not optional.

7.1 Runtime Isolation Technology Comparison

Technology	Isolation Model	Best Fit for Agent Workloads	Tradeoff
Firecracker microVMs	Hardware-virtualized, minimal device model, AWS-originated	High-trust-boundary agent code execution (e.g., user-submitted code agents); the de facto standard referenced by current MCP gateway guidance for "one ephemeral sandbox per task"	Higher cold-start latency than containers; requires KVM
Kata Containers	OCI-compatible container interface backed by lightweight VMs	Drop-in replacement for container-based agent runtimes needing VM-grade isolation without rearchitecting orchestration	Operational complexity of managing a hypervisor layer in a Kubernetes estate
gVisor	User-space kernel intercepting syscalls	General-purpose agent sandboxing where full VM isolation is unnecessary but syscall-level containment is required	Syscall interception overhead; some syscall compatibility gaps
WASM / WASI	Capability-based, sandboxed bytecode execution	Tool execution and untrusted plugin code where startup latency must be near-zero; increasingly used for MCP tool sandboxing	Language/runtime ecosystem still maturing for general-purpose agent code
Sandboxed Containers (gVisor/Kata aside, i.e. seccomp + AppArmor hardened standard containers)	Namespace and cgroup isolation with restricted syscall/capability surface	Lowest-overhead option for lower-trust-boundary agents; acceptable when paired with strict network egress controls	Weakest isolation guarantee of the options listed; shared kernel risk

The architectural pattern this program recommends is tiered: classify agents by the trust boundary they cross (does this agent execute model-generated code? does it process untrusted external content? does it have network egress to sensitive internal systems?) and assign isolation strength accordingly, rather than applying one isolation technology uniformly. A read-only reporting agent with no code execution does not need Firecracker-grade isolation; a coding agent executing arbitrary generated Python against production-adjacent infrastructure does.

7.2 Runtime Controls

- **Capability-based security** — agents hold unforgeable capability tokens scoped to specific resources and operations rather than ambient authority; possessing the capability is both necessary and sufficient, which makes scope review tractable in a way that role-based ambient permissions are not.
- **Dynamic permissions** — permissions are granted just-in-time for the duration of a specific task and revoked on completion, rather than provisioned as standing access — directly addressing the over-permissioning pattern that current NHI research identifies as the most operationally urgent gap in enterprise agent deployments.
- **Runtime authorization** — every tool call and action is re-evaluated against policy at the moment of execution (not only at session start), so a token issued for a benign initial task cannot be silently reused for an unrelated, higher-risk action later in the same session.
- **Action approval workflows** — irreversible or high-consequence actions route through a human approval gate or a secondary automated control (a policy engine, a separate verification agent) before execution, with the approval itself captured in the immutable audit log.

7.3 Secure Agent Operating System Pattern

Across vendors and frameworks, the architecture that is converging as best practice resembles a minimal operating system built specifically for agent execution, with four layers stacked between the agent's reasoning loop and the outside world:

1. **Agent** — the reasoning/planning loop itself, treated as untrusted input to everything below it.
2. **Sandbox** — the isolation boundary (Firecracker/Kata/gVisor/WASM per the tiering above) that contains what the agent can directly touch.
3. **Policy Engine** — the authorization decision point (commonly OPA or Cedar — see Volume 2) that every action must clear before reaching a tool.
4. **Tool Broker / Runtime Controller** — the only component with actual credentials to downstream systems; it receives a policy-approved request and a scoped, ephemeral credential, executes the call, and returns only the result, never the credential, to the agent.

The structural property worth emphasizing: the agent itself never holds standing credentials to anything. It holds, at most, a short-lived capability token whose actual exchange for a usable credential happens inside the tool broker, after a policy check. This single design decision — credential custody lives outside the agent process — eliminates an entire category of incidents where a compromised or manipulated agent simply exfiltrates whatever secrets it was holding.

Domain 20 — AI Security Mesh Architecture

As agent populations scale past a few dozen into the hundreds or thousands, point-to-point security controls between every agent, tool, and data source stop being tractable — the same inflection point that drove the industry from point-to-point service integration toward service mesh architecture a decade ago. The AI Security Mesh pattern applies that same architectural move to agentic systems.

20.1 The Five Mesh Planes

Mesh Plane	Function	Representative Technology Pattern
Identity Mesh	Issues and verifies workload identity for every agent, tool, and MCP/A2A endpoint uniformly	SPIFFE/SPIRE trust domain spanning all agent infrastructure (full detail in Volume 2)
Policy Mesh	Centralizes authorization decisions so policy is defined once and enforced consistently at every enforcement point	OPA or Cedar policy bundles distributed to sidecars/brokers at each agent and MCP gateway
Agent Mesh	Manages discovery, routing, and trust establishment between agents and agent-to-agent calls	A2A trust broker with signed Agent Card verification (full detail in Volume 2)
Memory Mesh	Governs how memory is shared, isolated, and classified across agents and sessions	Tenant-isolated vector stores and graph memory with provenance tagging (full detail in Volume 4)
Observability Mesh	Aggregates traces, logs, and behavioral signals across every plane into a unified view	OpenTelemetry-based tracing (OpenLLMetry conventions) feeding a central AI SOC (full detail in Volume 4)

20.2 Comparison with Service Mesh and Zero Trust

The AI Security Mesh is best understood as a service mesh's conceptual sibling, not its replacement — most enterprises will run both, with the AI Security Mesh as an agent-aware layer on top of (or alongside) an existing Istio/Linkerd-class service mesh.

Property	Traditional Service Mesh	AI Security Mesh
Unit of identity	Service / workload (often per-deployment)	Individual agent instance, often ephemeral and per-task
Primary trust question	Is this service authorized to call that service?	Is this agent authorized to take this action, with this data, on behalf of this principal, right now?
Policy granularity	Network-level (which services can talk to which)	Action-level (which specific tool calls, with which parameters, are permitted)

Property	Traditional Service Mesh	AI Security Mesh
Failure mode of concern	Service unavailability, latency, network partition	Goal hijacking, cascading multi-agent failure, autonomous over-reach
Relationship to SPIFFE/Zero Trust	SPIFFE commonly used as the identity substrate; zero trust is the governing principle	Same substrate (SPIFFE/SPIRE), extended with agent-specific attestation and behavioral trust scoring layered on top

Design recommendation

Do not build a parallel identity system for agents. Extend the organization's existing SPIFFE/SPIRE (or equivalent workload identity) trust domain to cover agent workloads, and layer agent-specific policy and behavioral-trust evaluation on top of that shared identity substrate. This is the path the most mature implementations (Block's production SPIFFE+WIMSE+OAuth deployment is the most frequently cited public reference) have taken, and it avoids the fragmentation that has plagued early agent-identity tooling.

Summary: How the Frameworks in This Volume Relate

Readers new to this space frequently ask which single framework to adopt. The honest answer is that no single framework covers the full problem, and the frameworks are not competitors — they operate at different altitudes and answer different questions.

Framework	Altitude	Primary Question It Answers
SABSA / TOGAF / Zachman	Enterprise architecture	How does agent risk and capability map onto our existing business and technology architecture?
STRIDE / PASTA	Application security	What conventional security flaws exist in the systems an agent runs on?
MITRE ATT&CK / ATLAS	Adversary tactics	What does a real-world attacker do once inside, and what AI-specific techniques apply?
CSA MAESTRO	Agent-native threat modeling	What can go wrong, layer by layer and across layers, in this specific agentic system?
OWASP ASI Top 10	Risk taxonomy / prioritization	Which of the highest-prevalence, highest-impact agentic risks are present here?
AI Security Mesh	Operating architecture	How do we enforce identity, policy, memory, and observability consistently at scale?

Volume 1 has established this scaffolding. Volume 2 goes deep on the identity, MCP, and A2A layers that this volume references but does not fully specify. Volume 3 covers the governance frameworks (OWASP, NIST, ISO 42001, regulatory) that determine which controls are mandatory versus discretionary. Volumes 4 through 6 cover operations, economics, and the roadmap for building this capability over 24 months.